Roadmap: Auth Scaffolding to Presentable

Prioritized execution plan to move BeltTrack from its current auth-scaffolding and interactive-preview stage to a demo-safe, presentable application.

Scope: current working tree at commit 19de146 | Audit date: November 2026 | Status: Planning (no files changed)

Definition of Presentable

Presentable means safe to demonstrate to a real dojo owner: core workflows persist to the database, tenant boundaries are enforced server-side, and there are no exploitable or fake-success paths. It does not require full production launch.

Current Verification Snapshot

- npm run audit: PASS - 45 files, 0 hardcoded color violations.
- npx tsc --noEmit: PASS - no type errors.
- npm run build: PASS - 15 routes compiled, with warnings (React 18 deprecated in Next 16, middleware convention deprecated, Turbopack root warning).
- Functional rating: 6/10 overall; 8/10 as an interactive demo; 3.5/10 as a production multi-tenant system.
- Core domain features (attendance, technique sign-off, ledger, inactivity alerts) are UI-only and not persisted. RLS is documented but not enforced.

Phase 0 - Security Blockers (must do first, ~1-2 days)

- Remove the hardcoded fallback password and all managed-auth-preview writes: login/route.ts:125, register/route.ts:157, join/route.ts:128,161, reset-password/route.ts:186. Fail closed with HTTP 503 when Supabase is unconfigured.
- Authorize the approvals route: require a session, verify OWNER or COACH, and derive schoolId from the DB user. Never trust a request body schoolId (approvals/route.ts:5-16, 69-117).
- Reduce the public directory to PRD fields only: name, slug, city, discipline. Remove joinCode and id (directory/route.ts:10-17).
- Fix the reset flow: remove the targetEmail echo, return generic responses, enforce the 15-128 character policy, and add strict per-email and per-IP limits (reset-password/route.ts:106-141).
- Add a server-side route guard and block PENDING_APPROVAL accounts from rosters, techniques, and the dashboard.
- Move verification and password-reset codes into PostgreSQL: hashed code, expiry, attempt counter, used flag, single-use. In-memory maps cannot work on serverless (email-verification.ts:29-34).
- Rotate Supabase service-role and SMTP credentials, confirm .env is not in git history, and add boot-time environment validation.

Phase 1 - Make the Demo Honest (persistence, ~3-5 days)

- Wire tenant enforcement with a withTenantContext helper that sets app.current_school_id and app.current_user_id, or implement a real Prisma $extends. Ship rls_policies.sql as a migration (fixes.md Steps 3-4).
- Attendance: add POST /api/attendance and persist dashboard check-ins. Coach-only, idempotency key, no backdating.
- Technique sign-off: add POST /api/techniques/verify with a server role check and persist TechniqueProgress.
- Ledger: add POST /api/ledger with a server-generated REC-YYYY-DISCIPLINE-N slug, append-only rows, and offsetting reversal entries (manual-ledger-modal.tsx:125-142).
- Inactivity alerts: run a real query for 10 or more absent days and remove the hardcoded Elena Rostova / 13 days block (preview-dashboard.tsx:686-693).

- Derive rank metrics (classes, days-in-rank, techniques) from the database per tenant. Keep 42/50, 142/120, and 8/10 only for the demo tenant.
- Persist country, drop or properly collect date of birth, and validate that guardian and student belong to the same tenant (join/route.ts:141-154).

Phase 2 - Presentability Polish (~2-3 days)

- Split Demo and Real modes with a visible banner. Remove simulated success toasts and fake data whenever a real session is active.
- Add a deterministic seed script for the demo tenant: school, belt ladder, techniques, class schedules, students, and ledger entries.
- Close UX gaps: loading, empty, and error states; mobile pass; no raw error.message in 500 responses.
- Housekeeping: move the inline head metadata into metadata exports (page.tsx:516-521), install ESLint and make npm run lint actually lint, and strip PII and session console logs.
- Upgrade to React 19, migrate middleware.ts to proxy, and set turbopack.root to remove build warnings.

Phase 3 - Production Hardening (if presentable means launchable)

- Replace the in-memory rate limiter with Redis or Upstash (requires package approval) and configure trusted proxy IP extraction.
- Add tests: Zod schema units, tenant-isolation integration tests, and e2e coverage for register -> join -> approve -> attendance -> sign-off -> ledger.
- Add security headers and CSP, CSRF tokens or strict same-origin enforcement, and an audit log for approvals and ledger mutations.
- Update the README to reflect actual status and add a deploy runbook plus environment checklist.

MINIMUM VIABLE PRESENTABLE SLICE

Ship Phase 0 plus one vertical slice (attendance + technique sign-off + ledger for a single real tenant) plus the demo seed and an honest demo banner. This converts the app from a mock dashboard into a credible working product without building everything.

Suggested Order

0 -> 8 -> 9/10/11 (vertical slice) -> 15/16 -> 12/13/14 -> 18/19 -> Phase 3 hardening.

Acceptance Gates Per Phase

- npm run audit reports 0 violations.
- npx tsc --noEmit is clean.
- npm run build is clean (no new warnings).
- Manual tenant-isolation test passes with two separate schools.

Assumptions

- Work is local only; no migrations, pushes, or commits without explicit instruction (AGENTS.md).
- Adding Redis or a test framework requires user approval first.
- Presentable targets a stakeholder or dojo-owner demo, not an open production launch.
- Estimates assume one developer familiar with the codebase.
- This document is a plan only; no application files were changed.